

DynamicSLM: AIKernel のための Capability モジュール型・自己改善型 Small Language Model

動的 Capability ロード、差分蒸留、異種計算実行のための Model ABI およびランタイムアーキテクチャ

Author: Takuya Sogawa

Affiliation: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: <https://doi.org/10.5281/zenodo.20550614>

Version: v0.1.0

Date: 2026-06-05

Status: Technical Note / Architecture Draft

License: CC BY 4.0

英語版を正本とし、日本語版は参考訳として含める。

Abstract

DynamicSLM は、AIKernel ランタイムがモデル能力を静的で不可分な巨大モデルの一部としてではなく、ロード可能で統治可能な Runtime Module として扱うための Capability モジュール型 SLM アーキテクチャである。本稿が扱う中心問題は、現在の LLM および SLM が大きな不可分成果物として配備され、一度学習されると単一の関数として振る舞うため、特定 Capability だけを実行時に追加、削除、失効、置換、スケジューリング、または特化することが難しい点にある。

本稿では、AIKernel のための Capability モジュール型・自己改善型 SLM アーキテクチャである **DynamicSLM** を提案する。DynamicSLM は、各モデル成果物を Semantic Profile、Capability Graph、Execution Profile、Lineage、Payload から成る AIKernel Model ABI によって定義する。これらの ABI 要素により、ランタイムは、モデルが何を解決する意図を持つのか、どの資源を要求するのか、どのような由来を持つのか、どの Capability Payload が物理化されるのか、そして実行がどのように統治されるべきかを、重みのロード前に検証できる。

主な新規性は、モデル Capability をモノリシックな重み配備から分離する点にある。AIKernel は、タスクに必要な最小 Capability 部分グラフを解決し、対応する Payload だけを動的にロードし、ReplayLog に基づく Semantic Trace として実行を記録し、検証済み Teacher Model の Trajectory を対象 Capability モジュールへ差分蒸留する。これにより、ローカル SLM はモデル全体を再学習・再配備することなく、Workload 固有の能力を段階的に獲得できる。

本稿は概念的・アーキテクチャ上の技術ノートであり、実証ベンチマークを提示するものではない。目的は、SLM を AIKernel 内部で統治可能、ホットスワップ可能、追跡可能、段階的に自己改善可能な Semantic Process として扱うための、OS レベルの Model ABI、Capability モジュール化戦略、蒸留ライフサイクル、ランタイムスケジューリング規律を仕様化することである。

Keywords

AIKernel; DynamicSLM; Small Language Models; Capability Graph; Model ABI; Semantic Profile; Execution Profile; Differential Distillation; ReplayLog; Dynamic Capability Loading; Heterogeneous Compute; Semantic Context OS; Self-Improving AI Systems.

1 Introduction

現在の LLM および SLM は、多くの場合、静的なモノリスとして設計される。一度学習されると、モデルは単一の巨大なパラメータ化関数として配備される。アダプタ、ルーティング層、外部ツール利用機構はモデルの振る舞いを修正できるが、モデル成果物そのものは依然として大きな不可分単位として扱われることが多い。実行時に特定の能力だけをロードし、単一の能力だけを置換し、狭い振る舞い領域だけを失効させ、ある Capability だけを更新することは難しい。

本稿における OS 比喩は、厳密にはアーキテクチャ上の比喩である。モデルが従来 OS の Process と完全に同一の性質を持つという意味ではない。むしろ、モデル Capability が、OS が実行コンポーネント、動的ライブラリ、資源制約付き Process を統治するように、外部 Runtime 境界によって宣言、受理、スケジューリング、隔離、スワップ、監査、失効されるべきである、という意味である。

この静的構造は、エッジ推論、メモリ制約の厳しい配備、個人化アシスタント、統治された AI ランタイムの要求と合わなくなりつつある。ソフトウェア工学では、OS は単一の巨大バイナリから、プロセス、動的リンクライブラリ、メモリページング、アクセス制御、資源スケジューリングを備えた構造へ発展した。一方で、モデル配備はしばしば、単一の成果物をロードし、現在のタスクに必要なかどうかに関わらずすべての能力が利用可能になる形式に留まっている。

AIKernel は、この問題を OS の観点から捉える。AIKernel では、モデルを最終的な実行主体としてではなく、Semantic Context、Capability Contract、Resource Profile、Deterministic Execution Trace によって統治されるプロセスとして扱う。この見方では、モデルは単なるブラックボックスのテンソル Payload ではなく、カーネル制御下で宣言、検証、スケジューリング、隔離、改善可能なランタイムモジュールであるべきである。

本稿は、AIKernel に統合される Capability モジュール型・自己改善型モデルアーキテクチャ **DynamicSLM** を提案する。DynamicSLM は、AIKernel Model ABI に準拠した Semantic Profile と Weight Payload の集合としてモデルを定義する。このアーキテクチャは、モデル Capability を静的な重み配備から分離し、Capability レベルのメタデータを実行前にカーネルへ公開する。

本技術ノートの貢献は以下の 3 点である。

1. モデルを Semantic Profile、Capability Graph、Execution Profile、Lineage、Payload の 5 要素で表す AIKernel Model ABI を定義する。
2. AIKernel がタスクに必要な最小 Capability 部分グラフを解決し、必要な Payload だけをロードする Capability レベルの動的ロードアーキテクチャを導入する。
3. 実行時に検出された Capability 不足を Teacher Model へ委譲し、検証済み ReplayLog を対象 Capability モジュールへ差分蒸留する自己改善ループを提案する。

目的はすべてのモノリシックモデルを置き換えることではない。大きく一貫したモデルが有効なタスクは残る。本稿の目的は、SLM を OS 管理プロセスに近づけるためのランタイムアーキテクチャ、すなわちモジュール化、失効可能性、スケジューリング可能性、ホットスワップ可能性、追跡可能性、対象限定の自己改善可能性を定義することである。

2 Related Work

DynamicSLM は、AIKernel を Semantic Context OS のための形式的ガバナンス基盤として定義する、公開済みの Trajectory Governance 論文を基盤としている [1]。本稿における Model ABI、Capability Graph、Differential Distillation、および異種計算スケジューリングは、その基盤上に位置づけられる新しいアーキテクチャ拡張として提示する。

モジュール型モデルの研究では、疎ルーティング、Mixture of Experts、アダプタ、低ランク適応などが検討されてきた。代表的な例やシステムとして、Sparsely-Gated Mixture-of-Experts、Mixtral、DeepSeek MoE、Phi-3.5 MoE 型のモジュール配備、LoRA、QLoRA などが挙げられる。これらはパラメータ効率を改善し、モデル内部の一部コンポーネントを選択的に活性化する可能性を示している。しかし、多くはモデルアーキテクチャ内部の最適化であり、外部ランタイムが Capability を第一級実行対象として検査、検証、受理、失効、スケジューリング、監査するための OS レベル契約を定義するものではない。

AIKernel が必要とする境界は異なる。カーネルは、モデル内部で確率的に Expert を選ぶだけでは不十分である。どの Capability がロードされるのか、その Semantic Profile は何か、どの資源を消費するのか、どのように派生したのか、どの Payload が物理的に展開されるのかを明示する契約が必要である。そのため DynamicSLM は、モデルのモジュール性を単なるパラメータ効率の問題ではなく、ランタイムガバナンスの問題として扱う。

Tool Use や Function Calling は、LLM が外部 Capability を利用するための有効な手法である。Toolformer、OpenAI Function Calling、LangGraph、LangChain Agents はこの方向の代表例である。しかし、それらはしばしばモデル成果物そのものを変更しない。ツールは外部に存在し、モデルは依然としてモノリシックな Planner または Controller として振る舞う。DynamicSLM は、Capability 構造の一部をモデル成果物自体に移しつつ、AIKernel の外部 Capability Registry によって統治する。

Recursive Self-Improvement、Continual Adaptation、Preference Optimization、Trajectory-based Improvement の研究は、モデルが自らの出力を評価し、時間とともに更新されるパイプラインを探索している。例として Self-Refine、Reflexion、DPO、ORPO、DeepSeek-R1 のような推論特化モデルにおける Trajectory Distillation、Anthropic 等の Frontier Model 研究における Recursive Self-Improvement の議論が挙げられる。しかし、その多くは Prompt Engineering、全体ファインチューニング、広範な Preference Optimization、または広範な Adapter 更新に依存する。DynamicSLM は、改善単位を宣言済み Capability ノードへ狭める。システムは Capability 不足を検出し、必要に応じて強力な Teacher Model へ委譲し、得られた検証済み Trace を最小の関連 Capability モジュールへ蒸留する。

近年の AI OS、Inference Serving、Agent Runtime には、Microsoft Semantic Kernel、NVIDIA NIM、Triton Inference Server、Modal、Ray Serve などの Runtime・Deployment 機構が存在する。これらは重要な基盤を提供するが、Semantic Profile、Execution Profile、Lineage、Payload を持つ Capability 統治済み Shared Library 的単位として Model Artifact を直接定義するものではない。

DynamicSLM は、AIKernel の決定論的ガバナンスと Semantic Context 分離を、モデルの重み管理そのものに適用する点で異なる。SLM は、Model ABI、Capability Graph、Lineage Chain、Resource-aware Loader によって統治される Shared Library 的コンポーネントとして再構成される。

3 DynamicSLM Architecture

DynamicSLM の中核は **AIKernel Model ABI** である。この ABI は、AIKernel がモデルをロードする前に推論・検証できる標準化インターフェースである。従来の配備では、モデルは主としてテンソル Payload、Tokenizer、Configuration File として扱われる。DynamicSLM では、Payload はより広い統治対象成果物の一部にすぎない。

DynamicSLM 成果物は 5 つの要素で構成される。

- **Semantic Profile:** モデルまたは Capability モジュールの Semantic Contract であり、入力前提、出力スキーマ保証、有界な Semantic Scope、想定タスクドメインを含む。
- **Capability Graph:** モデルが提供する Capability とその依存関係を表す有向非巡回グラフ。
- **Execution Profile:** メモリフットプリント、期待レイテンシ、Compute Shape、Accelerator 適性、Scheduling 制約などの実行時資源要件。
- **Lineage:** Artifact の生成経路を表す出所記録であり、親モデル、Teacher Model、蒸留に使用した ReplayLog、訓練設定、Artifact Hash を含む。
- **Payload:** 宣言された Capability を実現する Weight Tensor、Adapter、Quantized Block、Tokenizer Fragment、または実行可能 Model Segment。

これら 5 要素は、AIKernel と DynamicSLM Artifact の間にある ABI 境界をまとめて定義する。Semantic Profile は契約を、Capability Graph は依存構造を、Execution Profile は資源形状を、Lineage は出所証明を、Payload は物理化される Model Component を表す。

この構造により、AIKernel はモデルロード前に Fail-Closed できる。Capability を主張するモデルであっても、Lineage が無効である、Execution Profile が非互換である、Semantic Contract が壊れている、Payload Hash が検証できない場合、Loader は Weight をメモリに展開する前に拒否できる。

3.1 Capability Graph

Capability Graph は、モデル Capability を依存関係付き DAG として表現する。各ノードは Semantic Ability に対応し、各エッジは正しい実行に必要な依存関係を示す。例えば SQL generation Capability は、schema understanding、logical reasoning、structured output formation に依存し得る。

このグラフにより、AIKernel は Task Intent を最小 Capability 部分グラフへ解決できる。毎回モノリシックモデル全体をロードするのではなく、現在の意図に必要な Capability を特定し、関連 Payload のみを選択する。これは静的な全体バイナリ実行ではなく、動的リンクに近いモデルランタイムを実現する。

グラフはガバナンスにも使われる。Policy は read-only summarization を許可しつつ、system command generation を拒否できる。Capability が明示的ノードであるため、Policy Decision Point は生成開始前に危険な Capability ロードを拒否できる。

3.2 Semantic Profile

Semantic Profile は、Capability モジュールが満たすと主張する Contract を定義する。以下のような項目を含み得る。

- 受理可能な入力ドメイン
- 出力スキーマまたは構造化応答保証
- Semantic Bound または Task Ellipsoid
- Failure Mode と Required Fallback Behavior
- 他 Capability への依存仮定
- AIKernel Context および ReplayLog 形式との互換性

Semantic Profile は Prompt ではない。ランタイムへ公開される宣言的 Contract である。AIKernel はこれを用いて、Capability が現在のタスクに適格か、そして下流 Pipeline がその出力を消費できるかを判断する。

3.3 Execution Profile

Execution Profile は Capability の運用コストを宣言する。期待 VRAM 使用量、CPU/NPU 適性、Quantization Format、Tensor Shape、期待 Latency、Prefetch 要件などを含む。Runtime Scheduler はこの Profile を使って、Payload をどこでいつロードするかを決定する。

この Profile は特にエッジデバイスで重要である。小型デバイスは 7B 級モデル全体をロードできないかもしれないが、1.0B から 1.5B 程度の Capability モジュールと小さな Adapter ならロード可能な場合がある。DynamicSLM は、この判断をモデルロード前にカーネルへ見える形にする。

3.4 Lineage and Payload Verification

Lineage は出所と完全性を提供する。各 Capability Payload は、親モデル、Teacher Model、Distillation Corpus、ReplayLog 集合、Training Configuration、Output Artifact Hash を記録する Hash Chain に関連付けられる。Loader は Payload を受理する前に Lineage を検証する。

これにより、不正な Model Replacement を防ぎ、再現性を支援できる。Capability が予期しない振る舞いをした場合、AIKernel はどの Teacher、どの Data、どの Distillation Process から生成されたかを追跡できる。

4 Dynamic Distillation Pipeline

DynamicSLM は静的成果物ではなく、AIKernel ガバナンス下で進化するように設計される。この進化を駆動するのが **Differential Distillation Pipeline** である。

AIKernel がタスクを受け取ると、必要な Capability 部分グラフを解決する。現在のローカル SLM に必要 Capability がない、または必要な Semantic Profile を満たせない場合、システムはより強力な Teacher Model へタスクを委譲できる。この Fallback は不透明な外部呼び出しとして扱われない。Teacher の推論 Trajectory、出力、検証結果、実行 Context は、ReplayLog に基づく Training Material として記録される。

特定の Capability Gap について十分な検証済み例が蓄積されると、AIKernel はバックグラウンドで Differential Distillation を起動できる。システムはモデル全体を再訓練しない。Capability Graph 上の特定ノードを対象とする。例えば、特定ドメインの JSON 構造化における反復的失敗は、その Capability だけの小さな Adapter または Payload 更新を生成し得る。具体的な更新機構は LoRA 型 Adapter に限定されない。QLoRA 差分、通常の Adapter、Block-level Update、Segment-level Payload Replacement、量子化テンソルパッチ、または Model ABI が受理する別の Capability 単位 Artifact として表現できる。

得られた Capability モジュールは、新しい Semantic Profile、Execution Profile、Lineage を持つ Payload としてパッケージ化される。AIKernel は完全性検証、Validation、Policy Admission の後にのみ新 Artifact を登録する。登録後、そのモジュールは将来の Routing で利用可能になる。

これにより **Dynamic Self-Improvement** ループが形成される。

1. タスクが Capability を要求する。
2. ローカル SLM の Capability が不足または不十分である。
3. AIKernel が Teacher Model へガバナンス下で委譲する。

4. 成功した Trajectory と Output が ReplayLog Material として保存される。
5. 対象 Capability モジュールが差分蒸留される。
6. 新モジュールが Lineage Chain 付きで登録される。
7. 将来の類似タスクは Teacher Model への依存度を下げてローカルで処理される。

これはメモリキャッシュや Demand Paging の Semantic 版に近い。システムは反復的な Runtime Demand を観測し、不足 Capability を物理化し、将来の Fallback Cost を低減する。

5 Runtime Execution Model

DynamicSLM は、Process Management、Dynamic Linking、Memory Paging、Heterogeneous Scheduling といった OS 概念を言語モデル実行へ適用する。AIKernel Runtime は、単にモデルをロードして呼び出すのではない。Intent を解決し、Capability Contract を検証し、Capability 部分グラフを選択し、対応 Payload をロードし、利用可能な Compute Resource へスケジューリングする。

IPipelineOrchestrator は Execution Profile を用いて、Capability モジュールを CPU、GPU、NPU、または他の Accelerator へ配置する。**IModelVectorRouter** は Task Intent を候補 DynamicSLM モジュールへ対応付ける。**Registry & Loader** は Lineage を検証し、互換性を確認し、必要 Payload のみを物理化する。

5.1 Model Hot-Swapping

Model Hot-Swapping は DynamicSLM の中核である。従来の Runtime では、大規模モデルのロードとアンロードに秒単位の時間がかかり、すべての VRAM を消費し得る。DynamicSLM はロード単位をモデル全体から Capability Payload へ縮小する。Runtime は必要 Capability を Page In し、非アクティブな Capability を Page Out できる。

メモリ制約の厳しい環境では、これにより複数の論理的モデル Capability が、単一の物理 Accelerator 上で時間的に共存できる。Runtime は頻繁に利用される Capability を常駐させ、Pipeline Graph に基づいて予測 Capability を Prefetch し、優先度の低い Module を CPU または NPU へ Offload できる。

5.2 Trajectory Integration and Context Isolation

複数 Capability モジュールが同じタスクに関与する場合、その出力を制御不能な干渉なしに統合する必要がある。AIKernel は Semantic Storage Model を通じて統合を行う。Semantic Storage Model は決定論的な中間表現層として機能し、一時的な Module Output を、Addressing、Replay、Hashing、後続伝播が可能な安定した Semantic Artifact へ変換する。中間 Artifact は ReplayLog Entry として保存され、後続モジュールへ不変 Context として提供される。

これにより Module Execution が隔離される。ある Capability が別の Capability の内部状態を直接変更することはない。Capability は統治された Semantic Artifact を通じて通信する。この方式により、実行経路は監査可能になり、モジュール間の非決定的干渉は低減される。

5.3 Capacity Budgets and Scheduling

DynamicSLM の Scheduling は単純な FIFO ではない。Scheduler は Dynamic Capacity Budget、Latency SLA、Resource Pressure、Task Priority を考慮する。緊急の対話型タスクには即座に VRAM を割り当て、バックグラウンドの Differential Distillation はアイドル状態の CPU または NPU へ移せる。

IComputeShapeAdvisor は Hardware Load を監視し、Placement Decision を支援する。Scheduler はモデルを静的 Blob ではなく、統治された Runtime Process として扱える。

6 Conceptual Evaluation

本節は明示的に非実証的評価である。DynamicSLM はアーキテクチャ・パラダイムであり、完全な実証実装は進行中である。以下の評価は、アーキテクチャ分析、仮説的 Module Size、予想される Hardware Behavior、そしてアーキテクチャ上妥当と思われる Runtime Effect に基づく。測定済みベンチマークとしてではなく、概念的評価として読むべきである。

6.1 Memory Footprint Expectations

一般的な Capability モジュールが 1.0B から 1.5B パラメータ相当の Model Segment であると仮定すると、DynamicSLM はアクティブタスクに必要な Module だけをロードできる。モノリシックな 7B 級 SLM と比較して、Capability Graph の一部だけを活性化する Workload では、Peak VRAM 使用量を大きく削減できる可能性がある。アクティブ Capability ノード数に依存するが、50% から 75% の Peak Memory Footprint 削減はアーキテクチャ上あり得る仮説であり、実証済みの結果ではない。

6.2 Expected Swap Latency

PCIe Gen4 または Gen5 の帯域幅と、Graph-aware Asynchronous Prefetching を前提とすれば、適切なサイズの Capability Payload について、Page-in/Page-out Latency は数十ミリ秒の範囲に収まる可能性がある。この期待値は仮説的であり、Payload Size、Quantization Format、Host-device Transfer、Memory Pinning、Runtime Prefetch Accuracy に依存する。Prefetch が有効で Payload が小さい場合、Structure-Generation-Polish Workflow のような対話型 Agent Pipeline で許容可能と考えられる。

6.3 Differential Distillation Efficiency

Replay-based Distillation は、焦点を絞った適応が広範な再訓練よりも少ない Sample で狭い Capability を改善できる可能性を示している。DynamicSLM はこの考えを Capability Node レベルに適用する。Distillation Target が狭いため、General-purpose Model の再訓練よりも Data-efficient になる可能性がある。ただし、これは今後実証すべきアーキテクチャ上の仮説である。

6.4 Personalization Behavior

DynamicSLM は、ユーザー固有またはドメイン固有の適応を Capability Module として物理的に隔離する。これにより、Multi-user Adaptation のメモリ効率の高い経路が示される。ユーザーごとに Model 全体を複製・ロードする必要はなく、共通 Base 上に個別 Capability Payload を重ね、AIKernel が Policy に従ってスケジューリングおよび隔離できる。これは配備構造に関する概念的主張であり、測定済みの Multi-user Benchmark ではない。

7 Discussion

DynamicSLM は推論効率だけでなく、安全性とガバナンスにも利点をもたらす。Capability が明示的な Runtime Object であるため、AIKernel はロード前にそれらを統治できる。Policy Decision Point は、制限された System Command Generation のような危険 Capability を、Token 生成前に拒否できる。これは、禁止 Capability が Active Runtime Boundary に物理化される前に遮断するため、事後的な Prompt Filtering よりも強い防御線となる。

このアーキテクチャは監査可能性も高める。各 Capability Payload は Lineage を持ち、各実行は ReplayLog で記録される。タスクが失敗したり Capability が安全でない出力を生成した場合、システムはどの Capability が有効であったか、どの Parent Model がそれを生成したか、どの Distillation Trace が関与したか、どの Runtime Context がそれを呼び出したかを特定できる。

一方で Capability モジュール化にはトレードオフがある。Capability が細かすぎると、Context Fragmentation が生じ得る。その場合、モジュール間の Semantic Gap を埋めるための Orchestration Overhead が増大する。一部のタスクでは、細分化された Capability の組み合わせよりも、大規模な Monolithic Model の一貫した Global Reasoning State の方が高い精度を示す可能性がある。DynamicSLM はすべてのモデル配備方式の普遍的代替ではなく、Capability モジュール化された Workload のための Runtime Architecture として理解すべきである。

重要な今後の方向性は、Capability Graph の自己組織化である。現在の設計では、Differential Distillation の対象ノードは Heuristics または Governance Policy によって選択される。将来のバージョンでは、AIKernel の Semantic Compiler が ReplayLog を解析し、反復的 Capability Gap を検出し、新しい Capability Graph ノードを提案し、対応する Distillation Process を自動生成する可能性がある。

8 Conclusion

本稿では、AIKernel のための Capability モジュール型・自己改善型 Small Language Model アーキテクチャ **DynamicSLM** を提案した。この設計は、言語モデルを静的なモノリシックバイナリではなく、OS 的 Runtime に よって統治される動的プロセスとして再解釈する。

AIKernel Model ABI により、各 Model Artifact は Semantic Profile、Capability Graph、Execution Profile、Lineage、Payload を公開する。AIKernel は、どの Capability をロードすべきか、どこへスケジューリングすべきか、どのように Provenance を検証すべきか、実行をどのように記録すべきかを判断できる。Differential Distillation により、システムは検証済み Runtime Trace を対象限定の Local Capability Module へ変換し、大規模 Teacher Model への依存を段階的に下げることができる。

DynamicSLM は、モデルが単なる確率的生成器ではなく、Semantic Context Operating System における統治可能、追跡可能、ホットスワップ可能、自己改善可能な決定論的実行基盤の構成要素となる未来への一歩である。

9 References / 参考文献

- [1] Sogawa, T. (2026). AIKernel Trajectory Governance Model: A Kernel-Level Framework for Convergent Decision Control over Stochastic Language Model Inference. Zenodo.
DOI: <https://doi.org/10.5281/zenodo.20309510>
- [2] Sogawa, T. (2026). AIKernel Phase-2 Theory: Semantic Compilation Architecture. Zenodo.
DOI: <https://doi.org/10.5281/zenodo.20312092>
- [3] Sogawa, T. (2026). AIKernel Semantic DSL Compiler and Deterministic Agent Execution Architecture. Zenodo.
DOI: <https://doi.org/10.5281/zenodo.20534341>
- [4] Sogawa, T. (2026). AIKernel Hash-Anchored Trust Layer (HATL): A Hybrid Symmetric Ledger with Hash-Based Public Anchors. Zenodo.
DOI: <https://doi.org/10.5281/zenodo.20502685>